



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET	V2.0.6	DOWNLOADS	1.1K	LICENSE	MIT	GITHUB PAGES	ONLINE
-------	--------	-----------	------	---------	-----	--------------	--------

Analisi del Codice VB.NET

1. Scopo Generale

Il file **DBQry.vb** contiene due classi (`DBApri` e `DataGVLoad`) che forniscono un'interfaccia semplice e centralizzata per:

1. **Gestire le connessioni a un database SQL Server** tramite una stringa di connessione definita nel file di configurazione (`app.config` o `web.config`).
2. **Eseguire query SQL** (SELECT, INSERT, UPDATE, DELETE) e restituire i risultati in forma di `DataTable` , valore scalare o numero di righe interessate.
3. **Popolare un controllo** `DataGridView` con i dati ottenuti da una query.

In pratica, il codice è un “wrapper” di accesso ai dati che riduce la duplicazione di codice e centralizza la gestione degli errori.

2. Componenti Chiave

Classe	Funzione	Descrizione
DBApri	<code>GetConnection()</code>	Restituisce una nuova istanza di <code>SqlConnection</code> usando la stringa di connessione <code>MiaConnessione</code> dal file di configurazione.
	<code>GetDataTable(query, parameters)</code>	Esegue una query SELECT e restituisce i risultati in un <code>DataTable</code> . Accetta parametri opzionali.
	<code>ExecuteScalar(query, parameters)</code>	Esegue una query che restituisce un singolo valore (es. <code>COUNT(*)</code>).

Classe	Funzione	Descrizione
	<code>ExecuteNonQuery(query, parameters)</code>	Esegue una query che non restituisce risultati (INSERT, UPDATE, DELETE) e ritorna il numero di righe interessate.
DataGVLoad	<code>ApriDGV(dgv, query, parameters)</code>	Popola un <code>DataGridView</code> con i risultati di una query. Gestisce eccezioni mostrando un <code>MessageBox</code> .

Dettagli Implementativi

- **Imports :**
 - `System.Configuration` → per leggere le stringhe di connessione.
 - `System.Data.SqlClient` → per l'API ADO.NET di SQL Server.
 - `System.Windows.Forms` → per l'interfaccia grafica (`DataGridView`, `MessageBox`).
- **NotInheritable :** la classe `DBApri` è dichiarata `NotInheritable` (sealed) per evitare ereditarietà indesiderata.
- **Private Sub New() :** costruttore privato che impedisce l'istanza della classe, poiché tutti i metodi sono `Shared`.
- **Using :** blocchi `Using` garantiscono la chiusura e la liberazione delle risorse (`SqlConnection`, `SqlCommand`, `SqlDataAdapter`) anche in caso di eccezioni.
- **Optional parameters As List(Of SqlParameter) = Nothing :** permette di passare parametri SQL in modo sicuro (prevenendo SQL injection) o di ometterli se non necessari.
- **LogLeggiScrivi.ScriviLog :** chiamata all'interno di `GetDataTable` per registrare eventuali errori. (Non è definita nel file, ma presumibilmente è una classe di logging personalizzata.)
- **MessageBox.Show :** visualizza un messaggio di errore all'utente quando `ApriDGV` fallisce.

3. Flusso di Esecuzione

3.1. Ottenere una Connessione

```

1 Using conn = GetConnection()
2     conn.Open()
3     ...
4 End Using

```

- `GetConnection()` legge la stringa di connessione dal file di configurazione e crea un nuovo `SqlConnection`.
- `conn.Open()` apre la connessione al database.

3.2. Eseguire una Query

a) GetDataTable

1. **Creazione del comando:** `New SqlCommand(query, conn)`.
2. **Aggiunta di parametri** (se presenti):
`cmd.Parameters.AddRange(parameters.ToArray())`.
3. **DataAdapter:** `New SqlDataAdapter(cmd)` per riempire un `DataTable`.
4. **Riempimento:** `da.Fill(dt)`.
5. **Restituzione:** il `DataTable` viene restituito al chiamante.

b) ExecuteScalar

1. Simile a `GetDataTable`, ma invece di riempire un `DataTable`, si chiama `cmd.ExecuteScalar()` che restituisce il primo valore della prima riga del risultato.

c) ExecuteNonQuery

1. Simile a `ExecuteScalar`, ma chiama `cmd.ExecuteNonQuery()` che restituisce il numero di righe colpite dalla query.

3.3. Popolare un DataGridView

```
1 DataGVLoad.ApriDGV(dgvClienti, "SELECT * FROM Clienti")
```

Info

1. Verifica che `dgv` non sia `Nothing`.
2. Chiama `DBApri.GetDataTable(query, parameters)` per ottenere i dati.
3. Imposta `dgv.DataSource` al `DataTable` restituito.
4. In caso di eccezione, mostra un `MessageBox` con l'errore.

4. Possibili Miglioramenti e Errori

Problema	Descrizione	Suggerimento
Logging non definito	<code>LogLeggiScrivi.ScriviLog</code> è chiamato ma non è presente nel file.	Implementare o importare la classe di log
Gestione delle eccezioni in <code>GetDataTable</code>	L'eccezione viene loggata ma poi rilanciata (<code>Throw</code>).	Potrebbe essere utile gestire l'eccezione livello di chiamante o restituire un <code>DataTa</code>

Problema	Descrizione	Suggerimento
Parametri opzionali	Optional parameters As <code>List(Of SqlParameter) = Nothing</code> è accettabile, ma si potrebbe usare <code>IEnumerable(Of SqlParameter)</code> per maggiore flessibilità.	Cambiare il tipo a <code>IEnumerable(Of SqlParameter)</code> e usare <code>If parameters IsNot Nothing cmd.Parameters.AddRange(parameters.</code>
Stringa di connessione	Dipende dal file di configurazione; se la chiave <code>MiaConnessione</code> non esiste, si otterrà un errore.	Aggiungere un controllo di esistenza della chiave e un messaggio di errore chiaro.
Uso di MessageBox in libreria	L'uso di <code>MessageBox</code> rende la classe dipendente dall'interfaccia utente.	Separare la logica di business dalla UI: l'accesso ai dati dovrebbe lanciare eccezioni e la gestione dovrebbe gestirle.
Performance	Ogni chiamata apre e chiude una connessione.	Se si eseguono molte query in rapida successione, si può considerare l'uso di <code>SqlConnection pooling</code> .

Il codice fornisce un set di metodi statici per:

- **Connettersi** a SQL Server in modo centralizzato.
- **Eseguire** query di lettura, scrittura e recupero di singoli valori.
- **Popolare** controlli UI (`DataGridView`) con i risultati.

Conclusioni

La struttura è pulita, utilizza `Using` per la gestione delle risorse e supporta parametri SQL per sicurezza. Con qualche piccola ottimizzazione (logging, gestione degli errori, separazione UI/logic) il modulo diventa ancora più robusto e riutilizzabile.